ESP32 y los registros

Lo de AVR era DDR, PORT y PIN. En el ESP32 cambia el nombre, pero se sigue controlando el hardware escribiendo bits.

ATmega trabaja con 8 bits, ESP32 clásico 32. Tiene dos núcleos, más memoria y comunicación inalámbrica. El voltaje de los GPIO es 3,3 V.

La placa DevKit tiene el módulo WROOM, chip USB-serie (puede ser CP2102 o CH340), regulador, botón EN y BOOT. El WROOM no es lo mismo que toda la placa.

GPIO significa entrada/salida de propósito general. Con la GPIO matrix varias señales de los periféricos pueden dirigirse a diferentes pines. IO_MUX configura las funciones.

GPIO_ENABLE parecido al DDR para configurar salida. GPIO_OUT maneja lo que se escribe y GPIO_IN para leer.

W1TS = write 1 to set, pone a 1 los bits que le indico.

W1TC = write 1 to clear, limpia esos bits. Los ceros que escriba no afectan las otras posiciones.

Si uso leer-modificar-escribir un registro, puede venir otra tarea o interrupción y cambiarlo antes de que guarde. Ahí se pierde el cambio. W1TS y W1TC ayudan porque se escribe directamente la máscara.

Ejemplo GPIO16, máscara 1UL << 16. Para prender con GPIO_OUT_W1TS y para apagar con W1TC. Antes debe estar configurado como salida.

En Arduino digitalWrite hace todo más fácil, pero debajo llama otras partes hasta llegar al registro.

Cuidado con los pines

GPIO16, 17, 18, 19, 21, 22, 23 suelen servir para prácticas, viendo que estén libres.

GPIO34 al 39 solamente entrada y no tienen pull-up interna. Los 6 al 11 normalmente van conectados con la memoria Flash del módulo, no ocuparlos.

GPIO1 y 3 usados por UART0, por ahí puede ir el monitor serie o la carga.

Los strapping son pines que el ESP32 lee al encender. GPIO0 puede decidir modo de programación. También 2, 5, 12 y 15, tener cuidado con lo que se conecta.

Si un sensor mantiene una condición equivocada cuando el chip arranca

GPIO35 para botón sí se puede usar como entrada, pero necesita resistencia externa si quiero pull-up. GPIO18 para LED suele ser más sencillo.

La fuente USB de la placa puede ser 5 V, pero los GPIO NO toleran 5 V. Resistencias en los LEDs; el cálculo era R = (3,3 - voltaje del LED) / corriente.

Del reset al loop

Primero el arranque desde ROM y bootloader, luego inicia la aplicación. ESP32 con Arduino usa funciones del core y FreeRTOS por debajo.

setup() corre una vez, sirve para configurar pines, serial, etc. loop() se repite. Es un superlazo, no se termina al acabar la primera vuelta.

En una vuelta se leen entradas, se decide y se actualizan salidas. Y otra vez lo mismo.

Variable global conserva el dato mientras corre el programa. Local normal dentro de loop se vuelve a crear. Con static el valor queda guardado entre vueltas.

Ejemplo contador: global 1,2,3; local 1,1,1; static 1,2,3.

La CPU sigue trabajando aunque el loop casi no haga nada. Si se quiere ahorrar batería, modos de bajo consumo o deep sleep, dependiendo del equipo.

FreeRTOS organiza tareas. Si uso el core de Arduino, no debo poner otro app_main() porque puede chocar con lo que ya trae.

Para cargar el programa

Arduino IDE, instalar esp32 by Espressif Systems. Elegir placa y puerto. El cable tiene que pasar datos, no solo carga.

Si no aparece puerto revisar driver; si no conecta, BOOT y strapping. Brownout cuando la alimentación cae. El puerto también puede estar ocupado.

En Wokwi se puede practicar el Blink con GPIO16, pero la placa física puede cambiar en sus conexiones.

La configuración de entrada y salida depende del GPIO y del módulo que estoy usando.